Personalized Grocery and Diet Planning Application (NutriSpoke)

Course: CP317

Group Number: 3

Names:

Zulqarnain Ali
Fatima Afzal
Dezirae Asante
Toshani Settivari
Bassil Obeidi

Roles:

Q/A & Testing: Toshani Settivari

Product Owner(s): **Dezirae Asante, Fatima Afzal**

Software Engineers: Bassil Obeidi, Dezirae Asante

UI/UX Designers: Zulqarnain Ali, Fatima Afzal

Scrum Master*:* Bassil Obeidi

Table Of Contents:

1. Sprint Goal Page 3
2. Sprint Backlog Page 3-5
3. Implementation Progress Page 5-10
4. Demo Video Page 10
5. Testing and Review Page 11-12
6. Group Blog Page 12

1. Sprint Goal

The goal of Sprint 2 is to extend NutriSpoke from calculating nutritional targets to helping users act on those targets through personalized grocery planning and meal suggestions. Building on the Profile, Preferences, and Targets features completed in Sprint 1, this sprint focuses on integrating two new features into one continuous workflow.

The Personalized Grocery Plan (GROC-01) and Meal Suggestions (MEAL-01) stories are selected because they represent the next step in the user's experience. Nutritional targets provide a daily reference, but users still need to decide which foods to purchase and what meals to prepare. The grocery planner addresses this by generating a seven-day list that users can review and revise, while meal suggestions provide recipes with ingredients, preparation instructions, and estimated nutrition. Both features use saved user information and dietary preferences, while preserving the ability to generate suggestions using default targets when information is missing.

These stories advance NutriSpoke's goal of making personalized nutrition planning more accessible and practical. Implementing them together also allows the team to establish a shared, and responsible approach to accessing a large language model through the Gemini API. This includes supplying user information and clear instructions, requesting structured responses, and validating the results before displaying and saving them. This integration builds on Sprint 1's calculations and data to connect users' nutritional goals to everyday food choices.

2. Sprint Backlog

ID	GROC-01
Title	Personalized Grocery Plan
Points	13
Priority	High
Assigned Members	Bassil, Zulqarnain
Tasks	- Create the grocery planner page and connect it to the backend. - Retrieve saved user information and nutritional targets. - Integrate Gemini to generate personalized weekly grocery lists. - Validate and save generated plans and nutritional estimates. - Display grocery items and weekly nutritional summaries. - Add filtering and sorting controls. - Allow users to regenerate or revise a plan using feedback. - Allow users to record a purchase and save the grocery inventory.

	- Handle loading, empty states, and errors. - Connect page to existing workflow.
Status	Complete
Dependencies & Integration Notes	- Builds on the existing optional-input workflow, allowing suggestions to be generated from personalized targets when information is provided or default targets when fields are left blank. - Requires access to an LLM service (Gemini API key). - Extends the existing workflow. - Does not require meals to be generated, to generate a grocery list.

ID	MEAL-01
Title	Meal Suggestions
Points	13
Priority	Medium
Assigned Members	Toshani, Dezirae
Tasks	- Create the meal suggestions page and connect it to the backend. - Retrieve saved user information and nutritional targets. - Integrate Gemini to generate personalized meal suggestions. - Validate and save generated recipes and nutritional estimates. - Display meal details, ingredients, instructions, and nutrition. - Allow users to generate or regenerate suggestions. - Allow users to select and save meals. - Handle loading, empty states, and errors. - Connect page to existing workflow.
Status	Complete
Dependencies & Integration Notes	- Builds on the existing optional-input workflow, allowing suggestions to be generated from personalized targets when information is provided or default targets when fields are left blank. - Requires access to an LLM service (Gemini API key). - Extends the existing workflow.

	- Does not require a grocery list.

3. Implementation Progress

Features Implemented

During Sprint 2, our team extended NutriSpoke with a weekly grocery planner and a personalized meal suggestion feature. Both features build on the profile, dietary preferences, and nutritional target calculations developed in Sprint 1. Users can now use their saved information to generate a grocery list, request changes to that list, and explore recipes suited to their nutritional goals.

Both features access a large language model through the Gemini API using an API key obtained through Google AI Studio. The backend supplies the model with nutritional targets, dietary restrictions, and instructions defining the expected response. Each feature has a separate Python backend and Reflex interface, covering request preparation, response validation, JSON storage, and interactive display

The new pages follow the clean, minimal style established in Sprint 1, using consistent navigation, spacious cards, and shared buttons. Grocery summaries and meal cards group related information together, while colour-coded nutrient labels, consistent with sprint 1, make nutritional values easier to scan. Loading indicators, status messages, and clearly labelled controls help users understand the available actions and follow the progress of their requests.

Grocery Planner

The grocery planner generates a seven-day grocery list for one person using saved profile information, dietary preferences, and nutritional targets. To support integration, we extracted the get_targets() function from Sprint 1's target_calculations.py into its own script. This allows the targets page and both new planners to reuse a single function for calculating daily calorie, protein, carbohydrate, fat, and fibre targets. The grocery backend multiplies these values by seven and sends both daily and weekly targets to the model alongside saved allergies and dietary restrictions.

We defined a structured response containing each item's name, category, total weekly quantity in grams, preparation details, and estimated nutritional values. Preparation details specify whether quantities represent raw, cooked, dry, or drained food, while nutritional values describe the entire listed quantity. The backend validates the response, calculates weekly nutritional totals by adding the individual item values, and saves the suggestion as JSON.

Users can revise an existing plan by entering written feedback. The backend sends the current list, requested changes, and current targets and restrictions to the model. A separate revision prompt asks it to preserve unaffected choices where practical, make suitable substitutions, and adjust quantities across the complete list. A successful revision replaces the saved suggestion. Users can then select **Record purchase** to save a separate weekly grocery inventory snapshot.

The frontend places weekly nutritional totals and daily averages above the grocery list to provide an overview before users review individual foods. Each item displays its quantity, preparation details, and colour-coded nutritional estimates. Category filters and sorting controls help users organize the list.

```
GENERATION_SYSTEM_PROMPT = BASE_SYSTEM_PROMPT + """
Generate a new weekly grocery list from daily_targets, weekly_targets,
allergies, and diet_types. Select suitable foods and quantities to cover the
full week. Return the complete new list and explain assumptions in notes.
"""
REVISION_SYSTEM_PROMPT = BASE_SYSTEM_PROMPT + """
Revise current_suggestion by applying user_feedback. Start from the supplied
list and preserve unaffected food choices and quantities where practical.
Remove foods the user excludes, replace them with suitable alternatives, and
adjust quantities as needed to rebalance calories, macros, and fiber across
the whole week. Retain earlier food preferences recorded in the suggestion
unless the feedback explicitly changes them. The supplied current targets,
allergies, and diet_types remain binding even if the saved suggestion differs.
Return the complete revised list, not just the changes. Explain substitutions,
retained food exclusions, and any remaining target gaps in notes.
"""
```

Figure 1. Gemini system prompts defining the generation of new weekly grocery plans and the revision of existing plans based on user feedback.

```python
def _save_suggestion(response: dict) -> dict:
    suggestion = {
        "items": response["items"],
        "estimated_weekly_totals": {
            name: sum(item[name] for item in response["items"]) for name in NUTRIENTS
        },
        "notes": response["notes"],
    }
    _save_json("weekly_suggestion.json", suggestion)
    return suggestion
```

Figure 2. Backend implementation for calculating estimated weekly nutritional totals of the generated grocery list and saving generated grocery suggestions (distinct from adding to user's inventory; record_purchase()).

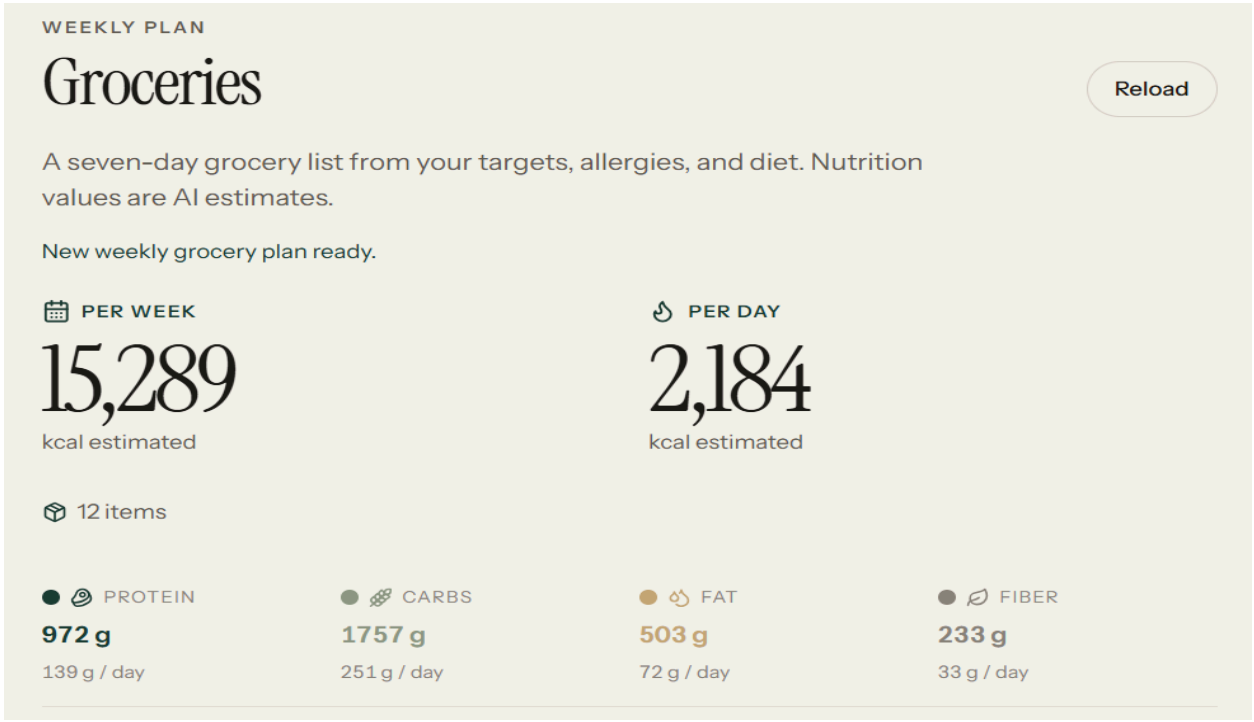


Figure 3. Weekly and daily estimates, with colour coded macro break-downs.

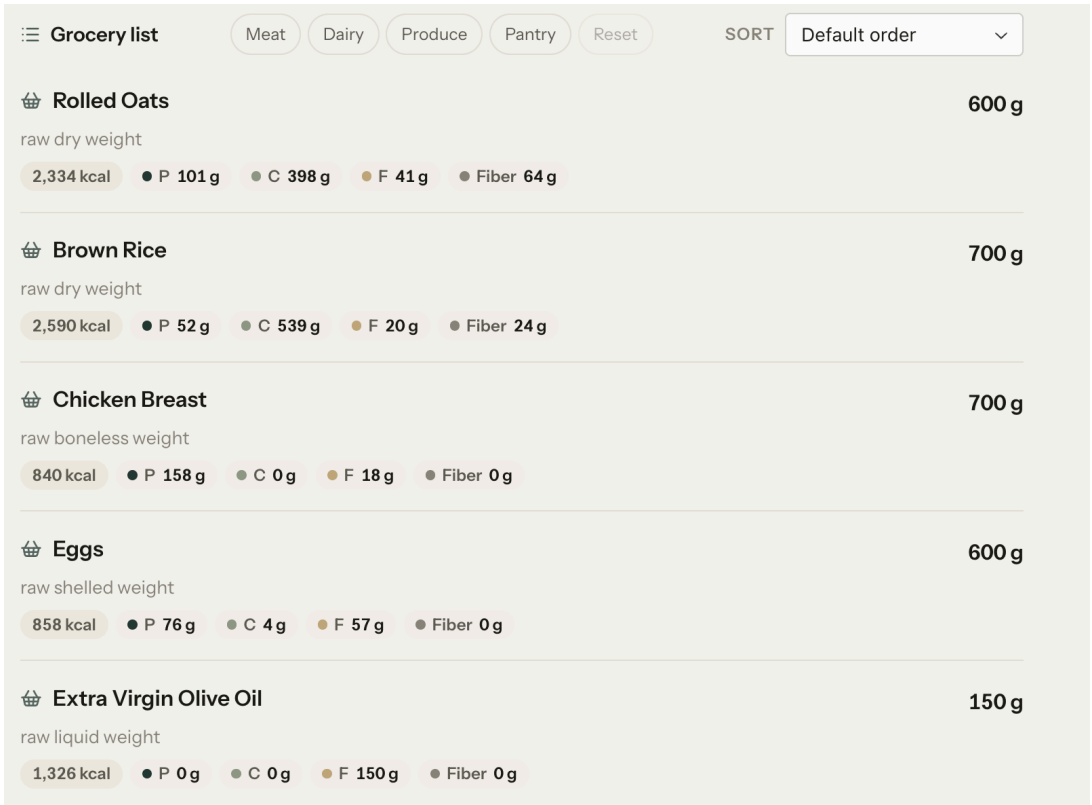


Figure 4. Example of grocery list generated based on profile and preferences.

Meals Generation

The meal suggestion feature uses the same saved profile, dietary preferences, and reusable get_targets function to generate personalized recipes. The application requests five suggestions by default, while the backend supports requests for between one and ten meals.

We defined a structured recipe response containing a name, description, meal type, serving count, ingredients, preparation instructions, and estimated calories, protein, carbohydrates, fat, and fibre. Supported meal types include breakfast, lunch, dinner, snack, and dessert. The prompt requests practical recipes with accessible ingredients and variety while treating dietary restrictions as mandatory. The backend validates the returned recipes before saving them as JSON.

Users can generate another set of suggestions or select individual meals to save for future planning. Selected meals are stored separately from generated suggestions, and the interface changes their buttons to indicate that they have been saved. When suggestions are loaded again, the page checks for matching selections and restores their saved indicators.

The frontend presents each recipe in a separate card, grouping its description, serving count, ingredients, instructions, and nutritional breakdown together. Consistent colour-coded nutrient labels connect the meal cards visually to the grocery page, and a label clarifies that nutrition estimates apply to the full recipe. Loading indicators, status messages, and saved buttons provide feedback as users generate and select meals.

```
"""

MEAL_GENERATION_PROMPT = BASE_MEAL_PROMPT + """

Generate personalized meal suggestions using the supplied:
- Daily nutrition targets
- Dietary goals
- Allergies
- Dietary preferences

Generate five different meal suggestions unless another number is specified.

Each meal must include:
- Name
- Description
- Meal type
- Number of servings
- Ingredients and quantities
- Preparation instructions
- Complete nutritional breakdown

Prioritize meals that help the user work toward their daily calorie and macronutrient targets.
Return the complete JSON response.

"""
```

Figure 5. Gemini prompt defining the requirements for generating personalized meal suggestions based on users' nutritional targets, dietary preferences, and restrictions.

```
MEAL_PROPERTIES = {
    "name": {"type": "string"},
    "description": {"type": "string"},
    "meal_type": {
        "type": "string",
        "enum": ["Breakfast", "Lunch", "Dinner", "Snack", "Dessert"]
    },
    "servings": {
        "type": "integer",
        "minimum": 1
    },
    "ingredients": {
        "type": "array",
        "items": {"type": "string"}
    },
    "instructions": {
        "type": "array",
        "items": {"type": "string"}
    },
    **{
        name: {"type": "number", "minimum": 0}
        for name in NUTRIENTS
    }
}
```

Figure 6. JSON schema defining the structure of AI-generated meals, including recipe details, serving information, and nutritional values.

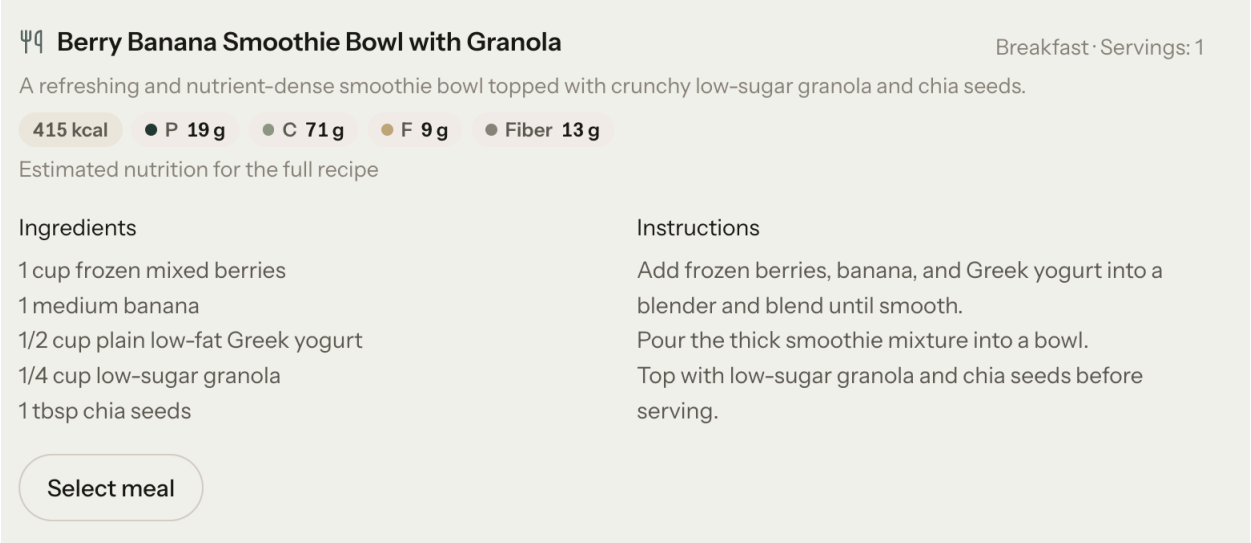


Figure 7. Example of Meal Generated based on goals and preferences.

Integration with Existing Modules

The new features were incorporated into one continuous workflow: users enter their profile information, set dietary preferences, review their nutritional targets, and then generate grocery lists and meal suggestions using that saved information. Both planners reuse Sprint 1's nutritional calculations and data files, so users do not need to enter their information again. Shared navigation, layouts, and controls keep the experience consistent across the application.

The meal page shares the grocery page's nutrient labels and model-unavailability detection, keeping presentation and error handling consistent.

Generated grocery and meal results are saved in separate JSON files within browser-session directories. Each page can load its session's saved suggestions, while purchased groceries and selected meals have their own files. Generation and grocery revision run in the background, with progress indicators and relevant controls disabled while requests are processing.

Challenges

We needed Gemini to return grocery lists and recipes in a format the website could work with. We gave it a JSON structure to follow and checked the results before saving them. For groceries, we checked things like whether each item had a valid category and a positive quantity. For meals, we checked that each recipe had a valid serving count, ingredients, and preparation instructions. The Python code for both features also checked for invalid nutrition values. We asked Gemini to consider ingredients in sauces and other prepared foods when following dietary restrictions. The website relies on the model to follow dietary restrictions, while backend validation checks the structure and numerical values of its response. Despite these measures, we did not implement an efficient way to independently verify the nutritional accuracy or dietary suitability of the suggested foods. The application therefore prominently disclaims that nutritional values and suggestions are AI estimations.

Another technical challenge we encountered involved the Gemini API occasionally returning 503 errors, indicating that the service was temporarily unavailable due to high demand. To address this, we implemented loading indicators and temporarily disabled the generate button while requests were processing, preventing users from accidentally submitting duplicate requests. We also implemented error handling to display an error message if a request failed, allowing users to retry once the service became available.

The team also had to learn how to configure and use Google AI Studio and the Gemini API. This included understanding how the variables.env file stores configuration values, such as the API key and selected model, and why secret keys should be kept out of source code and shared repositories. We also learned how request and rate limits can affect the application and how to switch the configured model when needed. This helped us connect the new features to Gemini and troubleshoot issues during development.

4. Demo Video

Present in an .mp4 video attached to this submission.

5. Testing and Review

Our team verified the functionality of NutriSpoke's grocery planning and meal suggestion features through manual front-end testing and back-end unit testing. Manual tests were used to simulate user interactions, including generating and revising a grocery plan, filtering and sorting grocery items, recording a purchase, generating meal suggestions, selecting meals and more. Acceptance criteria were checked by comparing the expected results of each user story with the actual behaviour of the application. A detailed list of all individual front-end and back-end tests can be found in the testing spreadsheet attached to this submission.

The manual testing showed that grocery plan generation, plan revisions, filtering, sorting and recording purchases met their acceptance criteria. Generated grocery lists and meal suggestions followed the user's selected allergies and diet types, and grocery totals stayed close to the user's daily targets. Meal suggestions displayed complete nutritional information, and selected meals remained saved after refreshing the page. Navigation testing showed that the Groceries and Meals pages, along with their Back buttons, matched their acceptance criteria.

Back-end functionality was also verified using unit tests across test_grocery_planner.py and test_meal_planner.py. These tests covered saving and loading user data, allergy and diet validation, calorie targets, AI response validation, weekly totals, plan revisions, recorded purchases, meal selection and browser sessions. The Gemini API was simulated in these tests so that no real requests were sent. All listed back-end unit tests passed upon the first test run.

Since all front-end and back-end tests passed upon the first test run, and the Product Owner verified all stories as completed, team review sessions focused on additional features and software design. For additional features, the team decided that the application will be revised in the next sprint to include more diet type options for religious dietary requirements, such as halal and kosher. For software design, the Product Owner indicated that an overall profile page is an additional requirement not covered by the user stories planned for the next sprint. This will be added to the required tasks for the next sprint.

Testing Summary:

Story	Status Verified by Product Owner(s)	Method of Checking Acceptance Criteria	Notes
GROC-01	Pass	Unit testing for grocery plan generation, revision, AI response validation, saved data and session handling.	All tests passed on the first round of testing.

		Manual testing for GUI functionality, filtering, sorting, revising, recording purchases, and allergy and diet compliance.	
MEAL-01	Pass	Unit testing for meal generation, AI response validation, meal selection and saved data. Manual testing for GUI functionality, nutritional information display, meal selection, and allergy and diet compliance.	All tests passed on the first round of testing.

A detailed testing spreadsheet is attached to this submission titled “Sprint_2_Testing.xlsx”.

6. Updated Team Blog

Present in an excel sheet attached to this submission.